☆ Git & GitHub Interview Q/A ☆

Section 1: Basics

Q1. What is Git?

Answer: Git is a distributed version control system (DVCS) that helps track code changes, collaborate with teams, roll back to older versions, and manage branching/merging.

Pro Tip: Always mention "distributed" — it highlights Git's offline capability.

Q2. What is GitHub?

Answer: GitHub is a cloud platform to host Git repositories and collaborate using Pull Requests, Issues, Actions (CI/CD), and project boards.

Pro Tip: Say Git = tool, GitHub = platform.

Q3. Common Git commands?

Answer:

- `git init` → Initialize a new Git repository
- `git clone <url>` → Copy repository from remote to local
- `git status` → Show modified/untracked files
- `git add .` → Stage all changes
- `git commit -m "msg"` → Save a snapshot with a message
- `git log --oneline` → Show commit history in short form
- `git branch` → List branches
- `git checkout -b <branch>` → Create + switch to a new branch
- `git merge <branch>` → Merge branch into current
- `git push origin main` → Push local commits to GitHub
- `git pull` → Fetch + merge changes from remote
- `git fetch` → Only download changes (no merge)

Pro Tip: In interviews, mention `git commit -am "msg"` as a shortcut for tracked files.

Q4. Difference between Git and GitHub?

Answer:

Git (Tool)	GitHub (Platform)
• It is a version control system.	• It is a cloud-based platform.
• Works locally on your machine.	• Hosts Git repositories online.
• Used to track changes in code.	• Used for collaboration and code hosting.
• Uses commands like commit, branch, merge.	• Provides features like PRs, Issues, Actions.
• No built-in collaboration features.	• Enables team collaboration & project management.

- Git → Local VCS tool that manages versions of code.
- GitHub → Cloud hosting service that extends Git with collaboration features.

Interview Edge: Mention alternatives like GitLab, Bitbucket.

Q5. What is a repository?

Answer: A repository is a project directory managed by Git, containing code, commit history, and branches.

- Local repository → Exists on your machine (git init).
- Remote repository → Hosted on GitHub for team collaboration.

Pro Tip: Interviewers like when you highlight the difference between local and remote repos.

Q6. What is a commit?

Answer: A commit is a snapshot of code at a point in time, identified by a unique SHA hash, author, timestamp, and commit message.

Pro Tip: A clear commit message helps track why a change was made.

≽ Section 2: Branching & Merging ≼

Q7. What is a branch?

Answer: A branch is a parallel line of development. It allows developers to build features without affecting the main branch.

Pro Tip: Mention how feature branches prevent breaking production code.

Q8. What is a merge conflict? How do you resolve it?

Answer: A merge conflict happens when two branches modify the same code section. Git cannot decide which change to keep.

Steps:

1. Open file and check conflict markers.
2. Decide correct changes (or combine).

Q15. Do we always need git add before commit?

Answer: No. For tracked files, you can commit directly with `git commit -am "msg"`. But for new/untracked files, `git add` is mandatory.

Q16. What is git stash and when is it useful?

Answer: `git stash` temporarily saves uncommitted work and clears the working directory.

Example: In the middle of a feature, you need to switch to main to fix a bug → run `git stash`, later use `git stash pop` to restore.

Q17. What is .gitignore?

Answer: A file that lists files/folders Git should ignore (e.g., logs, build files, environment secrets).

Q18. Difference between git reset, git revert, and git checkout?

Answer:

- **reset** → Moves HEAD pointer back, possibly deleting commits.
- **revert** → Safely creates a new commit that undoes changes.
- **checkout** → Switches branches or restores files.

Pro Tip: Say "I prefer revert in teams because it preserves history"

Section 5: Remote Operations

Q19. What is the difference between git pull and git fetch?

Answer:

- **fetch** → Downloads changes into remote-tracking branch (origin/main) but doesn't merge.
- **pull** → Fetch + Merge into local branch.

Q20. What is a remote in Git?

Answer: A remote is a reference to a hosted repository (usually on GitHub). Default name = origin.

Q21. How do you push a branch to GitHub?

Answer:

```
git push origin branch-name
```

For first time:

```
git push -u origin branch-name
```

(-u sets upstream tracking for future pushes.)

Q22. What is the difference between fork and clone?

Answer:

- Fork → Copies repository to your GitHub account.
- Clone → Copies repository to your local machine.

Q23. What is a Pull Request (PR)?

Answer: A PR is a request to merge code into another branch on GitHub. It allows code review, discussion, and approval before merging.

≒ Section 6: Tags & Releases ≒

Q24. What is a Git tag?

Answer: A tag is a label pointing to a specific commit, often used for releases (v1.0.0).

Q25. Difference between lightweight and annotated tags?

Answer:

- Lightweight → Simple pointer.
- Annotated → Stores metadata like tagger, date, and message (preferred for releases).

Q26. How do you create and push a tag?

Answer: `git tag -a v1.0.0 -m "First release"`

3. Save → git add → git commit.

Pro Tip: Mention that handling conflicts quickly is vital in team projects.

Q9. Difference between git merge and git rebase?

Answer:

- **Merge** → Combines histories, keeps non-linear commits.
- **Rebase** → Replays commits on another branch, making history linear.

Pro Tip: Teams often use merge for collaboration, and rebase for personal branches.

Q10. What is a fast-forward merge?

Answer:

- A fast-forward merge occurs when no new commits exist on the target branch, so Git simply moves the branch pointer ahead without creating a merge commit.

Pro Tip: Fast-forward keeps history simple, but doesn't show that a branch existed.

Q11. What is a detached HEAD?

Answer:

- Detached HEAD occurs when Git points directly to a commit instead of a branch.
- Commits made here won't belong to any branch unless you create one.

Pro Tip: Used for experiments or reviewing older commits.

Section 3: Commands

Q11. Common Git commands?

Answer:

- `git init` → Initialize a new Git repository
- `git clone <url>` → Copy repository from remote to local
- `git status` → Show modified/untracked files
- `git add .` → Stage all changes

- `git commit -m "msg"` → Save a snapshot with a message
- `git log --oneline` → Show commit history in short form
- `git branch` → List branches
- `git checkout -b <branch>` → Create + switch to a new branch
- `git merge <branch>` → Merge branch into current
- `git push origin main` → Push local commits to GitHub
- `git pull` → Fetch + merge changes from remote
- `git fetch` → Only download changes (no merge)

> Pro Tip: In interviews, mention `git commit -am "msg"` as a shortcut for tracked files.

Q12. What is the difference between git pull and git fetch?

Answer:

- `git fetch` downloads commits from remote into remote-tracking branches (like `origin/main`) but doesn't merge them.
- `git pull` does fetch + merge, updating your current branch automatically.

Q13. What is git stash?

Answer:

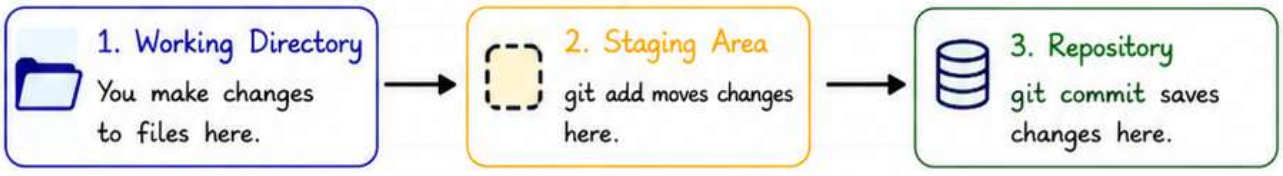
- `git stash` temporarily saves uncommitted changes and resets your working directory.
- Example: You are mid-feature but must switch to `main` to fix a bug. Run `git stash`, switch branch, fix and commit the bug, then return and restore with `git stash pop`.

Section 4: Staging & Workflow

Q14. What is git add and why do we need it?

Answer:

- `git add` stages changes from the working directory to the staging area.
- Git uses a 3-stage system: Working Directory → Staging Area → Repository



git push origin v1.0.0

Q27. How do you delete a tag?

Answer:

- Local : git tag -d v1.0.0
- Remote : git push --delete origin v1.0.0

Q28. How to see commits between two tags?

Answer: git log v1.0.0...v2.0.0 --oneline

Section 7: Advanced & Best Practices

Q29. What is GitHub Actions?

Answer: GitHub Actions is a CI/CD automation tool inside GitHub. It allows workflows like build, test, and deployment triggered on events (push, PR, etc.).

Q30. What is GitHub Flow vs Git Flow?

Answer:

- GitHub Flow → Simple strategy: create branch → PR → merge into main.
- Git Flow → Structured: uses main, develop, feature, release, hotfix branches.

Pro Tip: Mention GitHub Flow for startups, Git Flow for enterprise projects.

Q31. How do you handle large files in GitHub?

Answer: By using Git LFS (Large File Storage), which stores large binaries outside normal Git history.

Q32. What is the difference between main and origin/m/main?

Answer:

- main → Local branch.

Note:

- origin → Remote repository.
- origin/main → Remote branch named main.

- origin/main → Remote-tracking branch (state of remote main).

Q33. Best practices with Git in teams?

Answer:

- Write clear commit messages.
- Use branches for new features.
- Keep main protected.
- Resolve merge conflicts early.
- Use PRs for review.

Pro Tip: Add "We also use GitHub Actions for CI/CD" to show awareness of modern workflows.

@Theautomationengineer

Git-GitHub Cheatsheet

1. Basic Setup

- `git config --global user.name "Your Name"`
  `# Set your Git username.`
- `git config --global user.email "your.email@example.com"`
  `# Set your Git email.`
- `git config --list`
  `# List all Git configurations.`

2. Initializing and Cloning

- `git init`
  `# Initialize a new Git repository in your project.`
- `git clone <repo-url>`
  `# Clone an existing repository.`

3. Working with Changes

- `git add <file>`
  `# Stage a specific file for commit.`
- `git add .`
  `# Stage all changes in the current directory.`
- `git commit -m "Commit message"`
  `# Commit changes with a message.`
- `git commit -am "Message"`
  `# Add and commit tracked files in one step.`
- `git commit --amend`
  `# Edit the last commit message or add changes to it.`

4. Handling Merge Conflicts

- `git diff`
  `# Compare working directory changes.`
- `git diff <branch1> <branch2>`
  `# Compare two branches.`

`# Resolve conflicts:`
Open the files, fix conflicts,
then add and commit.

5. Status & Logs

- `git status`
  `# Show the current status of changes in the working directory.`
- `git log`
  `# View commit history.`
- `git log --oneline`
  `# Show concise commit history.`

6. Branching & Merging

- `git branch <branch-name>`
  `# Create a new branch.`
- `git checkout <branch-name>`
  `# Switch to a specific branch.`
- `git checkout -b <branch-name>`
  `# Create and switch to a new branch.`
- `git merge <branch-name>`
  `# Merge specified branch into the current branch.`
- `git rebase <branch-name>`
  `# Reapply commits on top of another base.`
- `git rebase -i HEAD~<n>`
  `# Interactive rebase to edit commit history, rearrange commits, modify commit messages, or squash the last n commits.`
- `git branch -d <branch-name>`
  `# Delete a local branch (use -D to force delete).`

★ By Achal Singh ☺ ★

@Theautomationengineer — By Achal Singh

Undoing Changes

- `git reset <file>`
  # Unstage a file.
- `git reset --soft HEAD~1`
  # Undo last commit but keep changes staged.
- `git reset --mixed HEAD~1`
  # Undo last commit, keep changes in the working directory (unstaged).
- `git reset --hard HEAD~1`
  # Completely remove the last commit.
- `git revert <commit-id>`
  # Create a new commit that undoes the specified commit.

Stashing Changes

- `git stash`
  # Temporarily save changes.
- `git stash list`
  # View stashed changes.
- `git stash pop`
  # Reapply stashed changes and remove them from the stash list.
- `git stash apply`
  # Reapply stashed changes without removing them.
- `git stash clear`
  # Remove all stashed entries.

Collaborating & Pull Requests

- `git branch -a`
  # List all branches, including remote.
- `git push origin :<branch-name>`
  # Delete a remote branch.

Creating a Pull Request: Go to your GitHub repository, select your branch, and click "New Pull Request."

Remote Repositories

- `git remote add origin <url>`
  # Link your local repository to a remote one.
- `git remote -v`
  # List the remote repository URLs.
- `git remote set-url origin <new-url>`
  # Update the remote URL for the repository.
- `git remote rename <old-name> <new-name>`
  # Rename a remote.
- `git push -u origin <branch-name>`
  # Push changes to the remote repository.
- `git pull origin <branch-name>`
  # Pull changes from the remote branch.
- `git fetch`
  # Download updates from the remote without merging.
- `git fetch <remote>`
  # Fetch updates from a specific remote.

Advanced Operations

- `git cherry-pick <commit-id>`
  # Apply a specific commit from another branch.
- `git cherry-pick <start-commit-id>^..<end-commit-id>`
  # Cherry-pick a range of commits.
- `git tag <tag-name>`
  # Add a tag to a commit.
- `git tag -d <tag-name>`
  # Remove a local tag.
- `git reflog`
  # View history of all changes (even uncommitted).
- `git reflog show <branch-name>`
  # Show reflog for a specific branch.
- `git show <commit-id>`
  # Show detailed info for a specific commit.
- `git bisect start`
  # Start bisecting to locate a bug.

Reviewing Changes

- `git show <file>`
  # Display changes made to a specific file.
- `git diff <commit-id1> <commit-id2>`
  # Compare changes between two commits.

Help Command

- `git help <command>`
  # Get detailed help for a specific command.

GitHub API (using curl)

- `curl -H "Authorization: token YOUR_TOKEN" https://api.github.com/repos/USERNAME/REPO_NAME/issues`

List issues in a repository.

Submodules & Worktrees

- `git submodule add <repo-url> <path>`

Add a submodule.

- `git submodule init`

Initialize submodules.

- `git submodule update`

Update submodules.

- `git worktree add <path> <branch>`

Create a new working tree for a branch.

Cleaning Up

- `git clean -f`

Remove untracked files.

- `git clean -fd`

Remove untracked files and directories.

- `git gc --prune=now`

Clean up unnecessary files and optimize the local repository.

GitHub Commands (Optional with GitHub CLI)

- `gh repo create`

Create a new GitHub repo from the command line.

- `gh repo clone <repo-url>`

Clone a GitHub repository.

- `gh pr create`

Create a pull request from the command line.

- `gh pr list`

List open pull requests in the repository.

- `gh issue create`

Create a GitHub issue from the command line.

Repository Management and Information

- `git shortlog -s -n`

Summarize commits by author.

- `git describe --tags`

Get a readable name for a commit.

- `git blame <file>`

Show who last modified each line of a file.

- `git grep "search-term"`

Search for a term in the repository.

- `git revert <commit-id1>..<commit-id2>`

Revert a range of commits.

- `git archive --format=zip HEAD -o latest.zip`

Archive the latest commit as a ZIP file.

- `git fsck`

Check the object database for integrity.

Best Practices and Common Workflows

- **Commit Often:** Make frequent commits with descriptive messages to maintain a clear project history.
- **Branch for Features:** Create a new branch for each feature or bug fix to keep changes organized and separate from the main codebase.
- **Use Meaningful Commit Messages:** Write clear and concise commit messages that explain the purpose of the changes.
- **Pull Regularly:** Regularly pull changes from the remote repository to stay updated with the latest changes and minimize merge conflicts.
- **Resolve Conflicts Promptly:** Address merge conflicts as soon as they arise to avoid complicating the integration process.
- **Review Pull Requests Thoroughly:** Ensure thorough review of pull requests to maintain code quality and facilitate knowledge sharing.
- **Tag Releases:** Use tags to mark important milestones or releases in the project for easy reference in the future.
- **Keep Your Branches Clean:** Delete branches that are no longer needed after merging them into the main branch to keep the repository organized.
- **Use Git Hooks for Automation:** Utilize Git hooks to automate tasks, like running tests before committing (pre-commit) or checking commit message formats. Hooks can help ensure code quality and consistency.
- **Squash Commits Before Merging:** Squash commits to combine related work into a single commit before merging, especially for feature branches. This keeps the project history clean and manageable.
- **Avoid Large Commits:** Try to keep commits small and focused on a single change or fix. This makes it easier to understand the history and isolate issues if something goes wrong.
- **Create Descriptive Branch Names:** Use branch naming conventions that describe the purpose, such as feature/login-form or fix/user-authentication-bug. This improves readability and collaboration.
- **Keep the Main Branch Deployable:** Always ensure that the main or production branch is stable and deployable. This allows the project to be released or updated at any time.

★ By Achal Singh ★